

1.如图箭头所指处的“pad_value=0”其中值取“0”是否合适？请说明原因。

答：合理。句子 A/B 的边界信息，来自序列本身的[SEP]，而非添加 segment_embedding[0]或者 segment_embedding[1]这种句子类型偏置。所以填充 0 是可以的。也就是说：token_type_ids 的 0/1 只决定“用哪一行 segment embedding”，并不决定“是不是多了一段句子 A/B”。句子边界本身是由 [CLS] 和 [SEP] 决定的，而不是由 0/1 决定的。

2.依据学习视频所述的 BERT 预训练的训练过程，还有什么地方可以进行优化？请尝试列出

答：

- Random_mask 写策略的时候，80% [MASK]，10% 随机替代，其余（10%）保持原样但不区分开，总体效果相近但不完全一致。对于保持原样不变的那 10%，被选中的位置要写上 labels[i]，即使不改 token。这样做使得预训练目标和论文更一致，mask 信号更丰富。
- 增加 attention mask，避免 PAD 干扰。现在 TinyBert 的 forward 里直接把 embedding 丢给 self.encoder(x)，没有传 src_key_padding_mask，结果就是 Self-Attention 会“看到” PAD 位置的表示，尽管 [PAD] 有单独的 embedding，也会引入一点噪声。改进方式是在 Dataset 里生成 attention_mask（非 PAD 为 1，PAD 为 0），然后在模型里构建 src_key_padding_mask = (input_ids == pad_id)，传给 encoder

3.在 BERT 的输入嵌入中，包括了三种 Embedding，分别是？



4. BERT 中 MLM 和 NSP 任务分别是怎样的？为什么 BERT 需要同时进行 MLM 和 NSP 任务的训练？单独训练其中一个任务的效果可能如何？

答：

- 上下文理解能力：通过掩码语言模型（MLM）任务，模型学会根据上下文预测被遮盖的词（类似“完形填空”）。示例：输入“I [MASK] an apple”，模型能预测“ate”。
- 逻辑推理能力：通过下一句预测（NSP）任务，模型能判断两句话是否为上下文关系。示例：判断“我饿了”和“我吃了一个苹果”是否相关。
- 【MLM 和 NSP 都要训练的原因】MLM 让模型学“局部+全局”的语言规律；NSP 让模型更关注跨句子的衔接和文档级信息，两者共享同一套 encoder 参数，能在有限语料上学到更泛化的表示。只训练“MLM”通常是够用、而且现在很常见（尤其在大规模预训练里），NSP 不是必须的；只训练 NSP 效果一般会很差，token 表示不够好，句子对表示也不够强，不推荐这么干。

5. BERT、GPT 与 Transformer 中的位置编码分别是怎样定义的？

- Transformer 位置编码：sin\cos 函数定义的

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- BERT 里面的位置编码：是绝对位置列表，在使用的时候直接查表就行。代码里面是 `nn.Embedding(12, hidden_dim)`
- 经典的 GPT 位置编码：也是绝对编码 `nn.Embedding(seq_len, embed_dim)` 做 position embedding。

6. BERT 的下游任务有哪些？分别如何设计？



BERT 典型的下游任务大致分 4 类，不同任务只是“在 BERT 顶上加一个很薄的头”，再用任务数据微调。

6.1 单句分类 (Sentence Classification)

- 例子：情感分析、主题分类、垃圾邮件检测等。
- 设计：
 - 输入：[CLS] sentence [SEP]
 - 用最后一层的 [CLS] 表示，通过一个 $\text{Linear}(\text{hidden} \rightarrow \text{num_labels})$ 做 softmax 分类。
 - Loss：普通交叉熵即可。

6.2 句对任务 (Sentence Pair Tasks)

- 例子：自然语言推理 (NLI, 如 MNLI)、语义相似度 (STS)、问句和候选答句匹配等。
- 设计：
 - 输入：[CLS] sentence_A [SEP] sentence_B [SEP]
 - token_type_ids: A 全 0, B 全 1。
 - 同样取 [CLS] 过 [Linear](#) 做分类或回归 (相似度可以用 MSE/回归 loss, 或者分桶后用交叉熵)。

6.3 序列标注 / Token 级任务 (Token-level Tasks)

- 例子：命名实体识别 (NER)、词性标注、槽位填充等。
- 设计：
 - 输入：[CLS] tokens... [SEP] (可以是一句或多句)。
 - 取每个 token 位置的 hidden state (一般忽略 [CLS]、[SEP])，接一个共享的 [Linear\(hidden \$\rightarrow\$ num_labels\)](#)。
 - Loss：对每个 token 做交叉熵 (可配合 CRF 进一步提升)。

6.4 抽取式阅读理解 / 问答 (Span-based QA)

- 例子：SQuAD 这类抽取式 MRC。
- 设计：
 - 输入：[CLS] question [SEP] passage [SEP]
 - 在 BERT 输出上接两个 [Linear](#)：一个预测每个 token 作为 start 的分数，一个预测 end 的分数。

- Loss: start 和 end 各一个交叉熵相加；推理时在合法范围内取最大 start_score + end_score 的 span。

7. GPT 的常用生成策略有哪些？分别进行阐述。

- 贪心解码：直接选概率最高的 token 作为生成的答案。缺点是容易每次回答都是单一词汇，陷入局部最优。
- 温度采样：除以温度参数再做 softmax，可以控制文本的保守度和多样性。
- Top-K: 先对总体排序，保留前 K 个；再对这保留的 K 个做归一化后做采样。
- Top-P: 先对总体排序，计算前面从大到小**累计概率总和为 p 的 token**，在此集合中采样
- 混合采样：温度采样+top k/p

8. 在视频“GPT 代码实现一文本生成”基础上调整生成策略，加入 Temperature Sampling。

```
def generate(model, tokenizer, prompt, max_len=50, strategy='greedy',
**kwargs):
    """
    自回归生成函数，支持 <bos>/<eos> 且屏蔽所有特殊 token。
    Args:
    model: 已训练好的 MiniGPT 模型
    tokenizer: SimpleTokenizer 实例，包含 special_tokens = ['[PAD]',
    '<bos>', '<eos>']
    prompt: str, 生成的起始文本（不含 <bos>）
    max_len: int, 最大生成长度（不包含 <bos>）
    strategy: 'greedy' | 'top_k' | 'top_p' | 'temperature'
    **kwargs:
        - top_k 时传 top_k=int
        - top_p 时传 top_p=float
        - temperature 时传 temperature=float (T>0, 越大越随机)
    Returns:
    List[str]: 生成的 token 列表（已去掉所有特殊 token）
    """
    device = next(model.parameters()).device
    # 特殊 token 的 id 列表
    special_ids = [tokenizer.vocab[t] for t in tokenizer.special_tokens]
    bos_id = tokenizer.vocab['<bos>']
    eos_id = tokenizer.vocab['<eos>']
    # 初始输入: 在 prompt 前加上 <bos>
    prompt_ids =
tokenizer.convert_tokens_to_ids(tokenizer.tokenize(prompt))
    x = torch.tensor([[bos_id] + prompt_ids], dtype=torch.long,
device=device) # [1, L+1]
```

```

for _ in range(max_len):
    # 只保留最后 block_size 长度以匹配模型输入限制
    x_cond = x if x.size(1) <= model.block_size else x[:, -
model.block_size:]
    # 前向，取最后一个时刻的 logits
    logits = model(x_cond)[0, -1] # [V] model(x_cond)-->[B,T,V]
    # 屏蔽所有特殊 token (包括 [PAD], <bos>, <eos>)
    logits[special_ids] = -float('inf')

    # 根据策略选择下一个 token id
    if strategy == 'greedy':
        idx = logits.argmax()
    elif strategy == 'top_k':
        probs, idxs = F.softmax(logits, dim=-
1).topk(kwargs.get('top_k', 10))
        idx = idxs[probs.multinomial(num_samples=1)]
    elif strategy == 'top_p':
        probs, idxs = F.softmax(logits, dim=-
1).sort(descending=True)
        cum = probs.cumsum(0)
        mask = cum <= kwargs.get('top_p', 0.9)
        probs = probs * mask
        idx = idxs[probs.multinomial(num_samples=1)]
    elif strategy == 'temperature':
        temperature = float(kwargs.get('temperature', 1.0))
        if temperature <= 0:
            raise ValueError(f"temperature must be > 0, got
{temperature}")
        probs = F.softmax(logits / temperature, dim=-1)
        idx = probs.multinomial(num_samples=1)
    else:
        raise ValueError(f"Unknown strategy: {strategy}")

    # 若生成 <eos>, 停止循环
    if idx.item() == eos_id:
        break
    # 拼接到输入序列
    x = torch.cat([x, idx.unsqueeze(0)], dim=1)

# 转回 tokens, 并过滤掉所有特殊 token
tokens = tokenizer.convert_ids_to_tokens(x[0].tolist())
return [t for t in tokens if t not in tokenizer.special_tokens]

```

9. GPT 模型架构基于 Transformer 的解码器实现，其注意力类型是自注意力还是交叉注

意力？

答：自注意力。在 `DecoderBlock.forward()` 里调用的是 `self.attn(x, x, x, attn_mask=mask)`，这里的 $Q=K=V=x$ ，所以是自注意力（Self-Attention）。

10. 近年来，BERT 启发了大量新模型（如 RoBERTa、ALBERT、ELECTRA、MacBERT、SpanBERT、DeBERTa）。任选一个你感兴趣的改进模型，比较它们相对于原始 BERT 的主要改进点和设计动机等变化。

答：

举例：RoBERTa（Robustly Optimized BERT Approach），它的核心思想不是改网络结构，而是证明 BERT 的很多“限制”来自训练配方不够强，改训练策略就能显著提升效果。

A. 相对 BERT 的变化总结：

- **结构层面**：基本不变（仍是 Transformer Encoder 堆叠）。
- **任务层面**：从“MLM + NSP” → 更偏向“只做强 MLM”（去 NSP）。
- **训练工程层面**：数据更大、训练更久、batch 更大、mask 更动态、长序列更充分。
- **结果直觉**：RoBERTa 说明“BERT 的潜力没被原配方用满”，很多提升来自更强的训练方案，而不是必须发明新结构。

B. 主要改进点（RoBERTa vs BERT）

- **去掉 NSP（Next Sentence Prediction）任务**
 - BERT 预训练有 MLM + NSP；RoBERTa 发现 NSP 对多数下游任务帮助不稳定，甚至可能干扰优化。
 - 动机：让模型把容量集中在更直接的语言建模信号（MLM）上，训练更稳、更有效。
- **更大的数据 + 更长的训练**
 - BERT 主要用 BookCorpus+Wiki；RoBERTa 用更多更杂的数据（如 CommonCrawl/News/OpenWebText 等），并训练更久。
 - 动机：Transformer 的表现高度依赖数据规模与优化充分性，增加语料和步数能显著提升泛化。
- **更大的 batch、更合适的学习率调度**
 - RoBERTa 强化了大 batch 训练（配合学习率/预热等策略）。

- 动机：提高训练稳定性与效率，减少噪声梯度带来的不确定性。
- **动态 Masking (Dynamic Masking)**
 - BERT 原实现常见做法是“静态 masking”（预处理时一次性生成 mask，整个训练反复用同一套 mask）。
 - RoBERTa 在训练过程中每次看到样本都重新随机 mask。
 - 动机：避免模型“记住”固定被遮盖位置，提高训练信号多样性，让 MLM 更接近真正的“填空”能力。
- **更长序列训练更充分**
 - BERT 虽支持 512，但训练时很大比例用较短序列；RoBERTa 更强调长序列阶段的训练占比/策略。
 - 动机：让模型更好利用长上下文能力，对长文本任务更友好。

11. 查阅最新资料，总结 GPT 的发展路线。

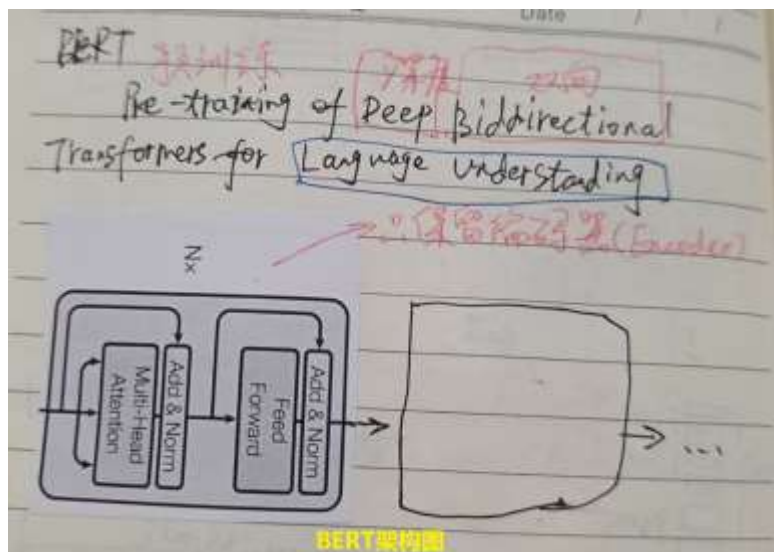
➤ **发展时间线**

- 2018 GPT：确立“生成式预训练 → 下游微调”的范式，把通用语言建模变成可迁移能力的来源。
- 2019 GPT-2：规模化 + 更强的零样本能力，同时开始把“安全发布/滥用风险”纳入发布策略。
- 2020 GPT-3：把“in-context learning (few-shot 通过提示学习)”变成核心用法：不更新参数，靠提示示例完成任务。
- 2022 InstructGPT：引入（并产品化）RLHF，让模型更“听指令、少跑偏”（对齐从“会做”走向“会按人类意图做”）：<https://openai.com/index/instruction-following/>
- 2022 ChatGPT：以对话为中心的交互范式落地，形成“持续迭代部署 + 反馈回路”的产品路径：<https://openai.com/index/chatgpt/>
- 2023 GPT-4：多模态（图文输入→文本输出）+ 更系统化的安全评估与红队流程：<https://openai.com/index/gpt-4-research/>
- 2023 Function calling：模型从“只会输出文本”走向“能输出结构化参数→调用工具/系统”的关键一步：<https://openai.com/index/function-calling-and-other-api-updates/>
- 2024 GPT-4o：端到端“全模态”（文本/音频/图像/视频任意组合）与低延迟语音交互，并配套系统卡与 Preparedness 评估框架：<https://openai.com/index/gpt-4o->

[system-card/](#)

- 2024 GPT-4o mini: 强调“高性价比 + 长上下文 + 强函数调用”，并引入分层指令等方式增强抗提示注入/越狱能力: <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>
- GPT-5: 强调“内置思考能力/更强编码与智能体任务、可控性与企业集成”等产品化落点: <https://openai.com/gpt-5/>
- **主线（从 GPT-1 到现在的演进方向）**
 - 能力来源: 从“预训练+微调”→“规模化+提示学习（in-context）”→“多模态统一建模”。
 - 可用性: 从“会说”→“会按指令说（RLHF）”→“会用工具做（函数调用/智能体链路）”。
 - 工程与安全: 从发布时讨论风险 → 系统卡/红队/Preparedness 这类“可评估、可部署”的治理与缓解机制并行推进。

12. 根据课程中 BERT 代码实现和 GPT 代码实现，绘制相应的模型架构图。



13. 从模型架构、训练任务、训练目标、训练样本构造、下游任务适应性、各自优势和

应用领域差异等方面对 BERT 与 GPT 进行全面对比分析。

- 模型架构

BERT: Transformer Encoder 堆叠（双向自注意力）

每个位置的表示可以同时看见左、右两侧上下文（bidirectional self-attention）。

典型用途：得到“整段文本/每个 token”的语义表示，偏理解/表征学习。

GPT: Transformer Decoder 堆叠（因果自注意力）

使用 causal mask，只能看见当前位置左侧上下文（left-to-right）。

典型用途：逐 token 生成，偏生成/建模条件分布。

- 训练任务与训练目标

GPT 学的是“顺序生成的真实过程”；BERT 学的是“缺词补全的语义一致性”

GPT: 因果语言模型

BERT: 掩码语言模型（Masked LM）+（早期常见）NSP

- 训练样本构造

BERT 输入构造（常见做法）

1. 加入特殊符号：[CLS]（句级表示）、[SEP]（分隔句子/段落）。
2. 句对任务：A 段 + [SEP] + B 段；并配合 segment embeddings（token type ids）区分句子 A/B。
3. MLM 的 mask 策略：例如 15% token 被选中，其中一部分替换成 [MASK]、一部分随机替换、一部分保持不变（为了减少“只在[MASK]上训练”的偏差）。

GPT 输入构造（常见做法）

1. 把文本当成长序列（可能跨文档，或按文档拼接），模型看到前缀，预测下一个 token。
2. 通常只需要 BOS/EOS 或简单分隔符；不需要 [MASK]、句对 segment id 这种结构（当然可以人为加入“提示模板”作为训练格式）。
3. 预训练时常用“滑窗/截断”形成固定长度 block；你之前在 miniGPT 里做的 CLM dataset 就是这个思路。

- 下游任务适应性

BERT 的典型下游范式：Fine-tuning 表征 + 小头

1. 句级分类：取 [CLS] 向量接分类器（情感、主题、自然语言推断等）。
2. 序列标注：每个 token 的输出接分类器（NER、POS、抽取式 QA 的 start/end）。
3. 检索/语义匹配：做 bi-encoder / cross-encoder（语义相似度、重排序）。
4. 优点：监督微调高效、稳定；在“理解型任务”上样本效率很好。

GPT 的典型下游范式：Prompting / Instruction-tuning / RLHF

1. 纯 prompt：用提示把任务“描述成生成问题”（zero/few-shot）。

2. 指令微调：把训练样本统一成“指令→答案”的生成格式，提高可用性。
3. 工具调用/函数调用：输出结构化参数，实现外部系统交互（更偏应用层能力）。
4. 优点：一个模型覆盖广泛任务形态（生成、对话、推理、代码），迁移方式更“统一”

● 各自优势（Strengths）与典型应用领域差异

要做 检索排序、文本匹配、信息抽取、分类 → 首选 BERT 系列（或 encoder 为主的架构）；

要做 写文章、写代码、对话、生成式总结、开放式推理 → 首选 GPT 系列（或 decoder 为主的架构）。

BERT 优势

1. 理解类任务强：分类、抽取、匹配、检索、结构化信息抽取等。
2. 输出可控：下游头是明确的分类/回归目标，少“胡说八道”的生成风险。
3. 计算特性：对固定输入做一次前向即可（不需要逐 token 解码），在很多判别式任务上推理更高效。

GPT 优势

1. 生成类任务强：写作、摘要、对话、翻译、代码生成、开放式问答。
2. 统一接口：很多任务可用“自然语言指令”直接完成，少改模型结构。
3. 可扩展到智能体：多轮规划、工具调用链、长文本合成等（配合外部系统）。